

internal GC. It is added as an isolation for GCs at build-time, which helps in performance.

Parallel full GC is implemented to remove the concurrent access of the GC. The normal G1 garbage collector is designed to avoid full collections, but when the concurrent collections can't access memory that fast, then an error, 'GC Full', will occur. Henceforth, in Java 10, parallel threads will be used to execute GC commands.

3. **Application class-data sharing:** This implementation in Java 10 is also to improve performance by sharing metadata across different Java processes. It extends the existing class-data sharing to allow application classes to be placed in the shared archive, to reduce the footprint and the startup time.
4. **Thread-local handshakes:** A handshake operation is a callback executed for each thread in Java when that thread is in a safe state (safepoint). The thread-local handshake introduces a way to execute a callback on threads without performing a global VM safepoint, which will make the process for stopping individual threads possible and inexpensive. Currently, this is not feasible for all architectures. Now, it has been implemented only for x64 and SPARC processors.
5. **Remove the native-header generation tool:** Javah, which is the functionality to write native header files at the time when Java source code is compiled (thereby eliminating the need for a separate tool), has been removed. This has been done because there were no direct JDK dependencies on the Javah tool.
6. **Additional Unicode language-tag extensions:** This feature adds some more Unicode language tags. It enhances the *java.util.Locale* package and the related APIs to implement additional Unicode extensions of the 'BCP (Best Current Practice) 47' language tags.
7. **Heap allocation on alternative memory devices:** In this feature, the focus is on heap memory allocation. Sometimes, because of multiple tasks running at the same time, heap memory gets full and leads to an error. To solve that problem, heap memory can now be allocated on other memory devices such as an NV-DIMM.

To allocate the heap in an alternative memory, add the following new option:

```
XX:AllocateHeapAt=<path>.
```

This option will map the memory to the path of the file system, and use memory mapping to achieve the desired result of allocating the object heap on the memory device.

8. **Experimental Java-based JIT compiler:** For experimental purposes, a new JIT (Just in Time) compiler has been introduced — Graal, which is to be used on the Linux/x64 platform. Graal is based on the experimental Ahead-of-Time (AOT) compiler introduced in JDK 9.
9. **Time-based release versioning:** A version number is a

non-empty sequence of elements separated by period characters (U+002E). A version number never has trailing zero elements. This is based on semantic versioning, which defines version numbers and the way they change, conveying meaning about the underlying code and what has been modified from one version to the next.

The versioning format in Java 10 is used like version number, *\$VNUM*, possibly followed by pre-release, build and other optional information, as shown below:

```
$VNUM(-$PRE)? \+$BUILD(-$OPT)?
$VNUM-$PRE(-$OPT)?
$VNUM(+-$OPT)?
```

The pros and cons of Java 10

In this new release from Oracle Corporation, many new changes have been made. These changes have been primarily done at the JDK level to boost performance by providing alternative heap memory, the thread-local handshake, etc. They will not have much of an effect at the code level.

When refactoring HotSpot internal code, there is a risk that performance could be harmed, for example if additional virtual calls are introduced. But this risk can be mitigated by continuous performance testing. It will break the work based on the assumption that nothing in the fundamental design of G1 prevents a parallel full GC. The fact that G1 uses regions will most likely lead to more wasted space after a parallel full GC, than for a single threaded one.

What's in it for developers?

For developers, the main change is the new local data type 'var'. This new feature will add some syntactic sugar to Java, simplifying it and improving the developer experience. The new syntax will reduce the verbosity associated with writing Java, while maintaining the commitment to static type safety. Developers frequently complain about the degree of boilerplate coding required in Java. Manifest type declarations for locals are often considered unnecessary; hence, using var is much better compared to declaring any other data type. Local variable type inference allows a similar effect in less tightly structured APIs.

Ever since the release of Java 9 last September, the whole platform has changed. Java 9 introduced us to modular Java and many other features, as well as the idea that Java would be switching to a six-month release cycle. Java 10 perhaps doesn't offer the biggest and most game-breaking changes, but it is the start of a new era of Java that promises a release every six months. **END** 🐧

By: Astha Srivastava

The author is a software developer, and can be reached at asthasri25@gmail.com.